

HERDWATCH
AI-Based Cattle Monitoring System



UNIVERSITY OF INFORMATION TECHNOLOGY

Group - 4
SEPTEMBER 2026

Course Name: CS-8117 Computer Vision

Project Title: HerdWatch AI-Based Cattle Monitoring System

Group Members

1. Yan Lin Naing

- Student ID: TNT-1934
- Email: yanlinnaing@uit.edu.mm

2. Khant Khant Wadi Hlaing

- Student ID: TNT-1977
- Email: khantkhantwadihlaing@uit.edu.mm

3. Thein Min Htet

- Student ID: TNT-1989
- Email: theinminhtet@uit.edu.mm

4. Mi Saung Wadi Zan

- Student ID: TNT-2005
- Email: mihsaungwadizan@uit.edu.mm

5. Thant Zin Moe

- Student ID: TNT-2081
- Email: thantzinmoe@uit.edu.mm

Supervisor: Dr. Thiri Mar Lar Swe

Department: Faculty of Computer Science

Submission Date:

Contents

1.Introduction.....	1
1.1 Overview.....	1
1.2 Project Scope.....	1
1.3 Objectives.....	2
2.Project Planning and Requirement Analysis.....	3
2.1 Functional vs. Non-Functional Requirements.....	3
2.2 Use Cases and User Stories.....	4
2.2.1 Use Case.....	4
2.2.2 User Stories.....	5
3.System Design and Architecture.....	6
3.1 System Architecture Design.....	6
3.2 System Process Flow.....	6
3.3 Main System Components.....	7
3.4 Database Design: ER diagrams.....	8
4.1 Tools and Technologies.....	9
4.2 Development Environment.....	9
4.3 Coding Standards.....	10
5. Screen Layout and Design.....	11
5.1 Dashboard.....	11
5.2 Cattle Monitor Screen.....	11
5.3 Main Interface Functions.....	12
6. Conclusion and Reflection.....	13
6.1 Project Summary.....	13
6.3 Lessons Learned.....	14
6.4 Future Enhancements.....	14
7. Appendices.....	15
Appendices A - Screen and Detection output.....	15
Appendices B - Key Project Source Structure.....	16
Appendices C - References.....	16

1.Introduction

1.1 Overview

HerdWatch Vision: Cattle Monitor is a computer-vision-based application designed to monitor cattle from images, recorded videos, and a live camera. The project combines a React web interface, a FastAPI backend, and a YOLO11s-based cattle detector. The system can detect cattle, maintain stable tracking identities in video, count visible animals within a configurable region of interest (ROI), and provide simple movement observations.

The project is intentionally positioned as an observation and monitoring tool rather than a veterinary diagnostic system. The user interface explicitly describes the conditions as visual observations and warns that persistent abnormal behaviour should be reviewed by a qualified veterinarian.

The supplied project contains separate frontend, backend, and computer-vision components. The frontend provides Dashboard and Cattle Monitor pages; the backend exposes health and prediction functionality; and the core vision module performs detection, tracking, counting, and movement-condition assessment.

1.2 Project Scope

- Accept cattle images and common video formats including JPG, PNG, BMP, MP4, AVI, MOV and MKV through the web interface.
- Support a live camera mode for continuous observation.
- Detect cattle using a trained YOLO11s detection model stored as `cattle_count_best.pt`.
- Restrict counting and movement observations to a configurable ROI.
- Use configurable detection confidence, line orientation, line position and ROI boundaries.
- Track cattle across video frames using ByteTrack and stable track IDs.
- Count animals using line-crossing logic for video and visible-in-ROI counting for static images.
- Classify observable movement into OBSERVING, LOW MOVEMENT, NORMAL MOVEMENT and HIGH ACTIVITY states.
- Produce annotated prediction images/videos and JSON metric artifacts.
- Provide system-health information indicating whether the API and trained detection model are available

Out of scope: the supplied implementation does not establish veterinary diagnoses, and the pure cattle-detection model does not provide a clinical lameness diagnosis. The source code explicitly notes that keypoint-based lameness detection is not available for the pure detection model.

1.3 Objectives

1. Develop a practical computer-vision application for cattle detection and monitoring.
2. Improve herd counting by combining object detection, ROI filtering, tracking IDs and line-crossing logic.
3. Provide a usable browser-based interface for uploading footage or using a live camera.
4. Give users interpretable movement signals without presenting them as medical diagnoses.
5. Separate frontend, backend and model-processing responsibilities so the application can be maintained and extended.
6. Store model outputs and job-level metrics so prediction results can be inspected after processing.

2. Project Planning and Requirement Analysis

2.1 Functional vs. Non-Functional Requirements

The following requirements are derived from the supplied source code and interface text. They describe what the current project is designed to do rather than unverified performance targets.

ID	Type	Requirement	Implementation Evidence
FR1	Functional	Upload cattle images/videos.	React Cattle Monitor upload control and accepted image/video types.
FR2	Functional	Run cattle detection.	YOLO model loaded from outputs/models/cattle_count_best.pt.
FR3	Functional	Track cattle in video.	Ultralytics tracking with ByteTrack and persistent track IDs.
FR4	Functional	Count cattle.	ROI visible counting for images and directional line crossing for videos.
FR5	Functional	Configure ROI and counting lines.	Frontend controls for orientation, position, confidence and x1/y1/x2/y2.
FR6	Functional	Assess movement observations.	Movement score based on recent tracked points.
FR7	Functional	Expose API health status.	FastAPI /health endpoint reports API status and model availability.
FR8	Functional	Save prediction artifacts.	Outputs directories contain prediction images and job metric JSON.

NFR1	Non-functional	Maintainable separation of concerns.	Frontend, backend, core vision and scripts are separate directories.
NFR2	Non-functional	Deterministic training configuration.	Training code sets seed=42 and deterministic=True.
NFR3	Non-functional	Input validation.	Counting configuration validates ROI, line position, hysteresis, minimum frames and confidence.
NFR4	Non-functional	Browser-compatible video output.	Video processing writes H.264 MP4 using imageio/FFMPEG.
NFR5	Non-functional	Safe interpretation.	UI warns that movement observations are not veterinary diagnoses.

2.2 Use Cases and User Stories

2.2.1 Use Case

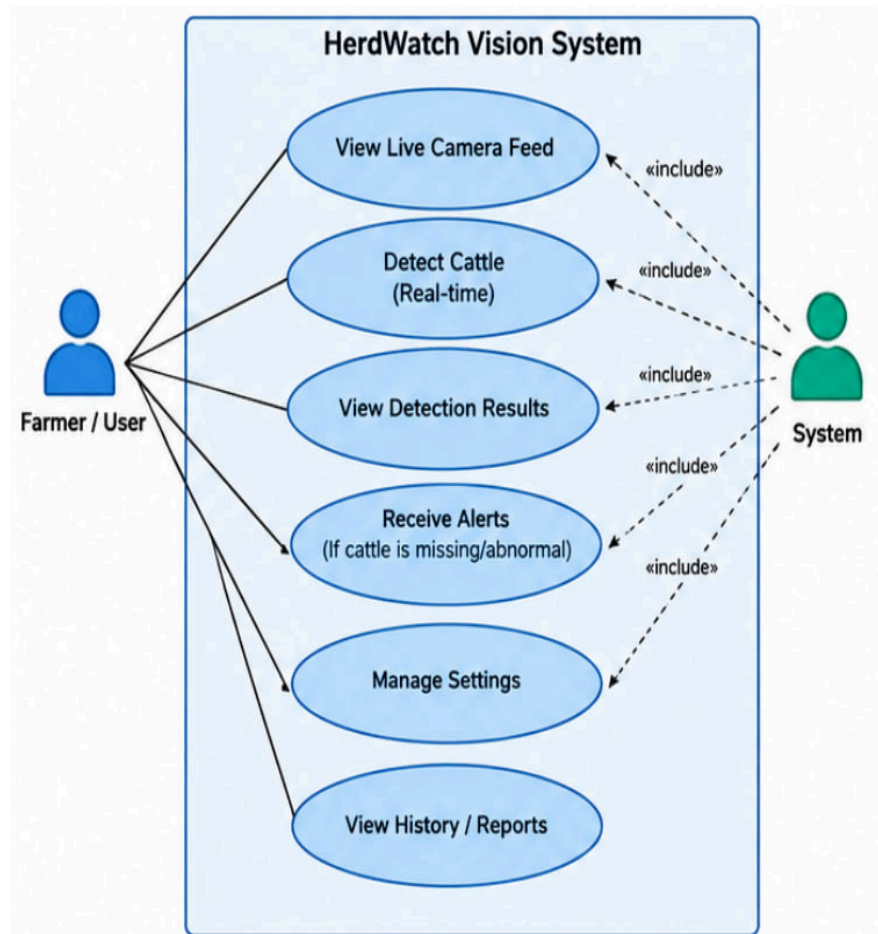


Figure 2.2.1 HerdWatch use-case diagram

2.2.2 User Stories

- As a farmer, I want to upload cattle footage so that I can automatically detect and count cattle.
- As a farmer, I want to use a live camera so that I can monitor cattle while the camera is running.
- As a farmer, I want to configure an ROI so that I can focus counting on a selected area.
- As a farmer, I want line-crossing counts so that I can monitor cattle moving through a boundary.
- As a farmer, I want movement status so that I can identify animals that may need closer observation.

3.System Design and Architecture

3.1 System Architecture Design

The system follows a layered architecture. The browser is responsible for user interaction and camera capture. The React frontend communicates with the FastAPI backend. The backend manages uploaded files, invokes the prediction core, performs live WebSocket inference, and exposes generated outputs. The trained YOLO model performs cattle detection, while OpenCV and NumPy support frame and trajectory processing.

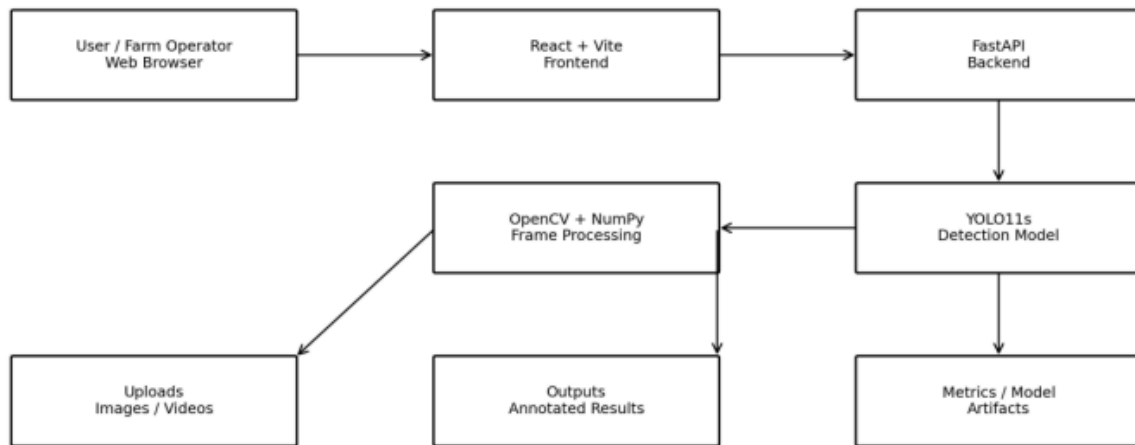


Figure 3.1: System Architecture Design

3.2 System Process Flow

The process begins with an image, uploaded video, or live camera frame. The frame is prepared for AI processing, YOLO11s detects cattle, ByteTrack associates detections between frames, and the system performs counting and movement analysis before displaying the result.

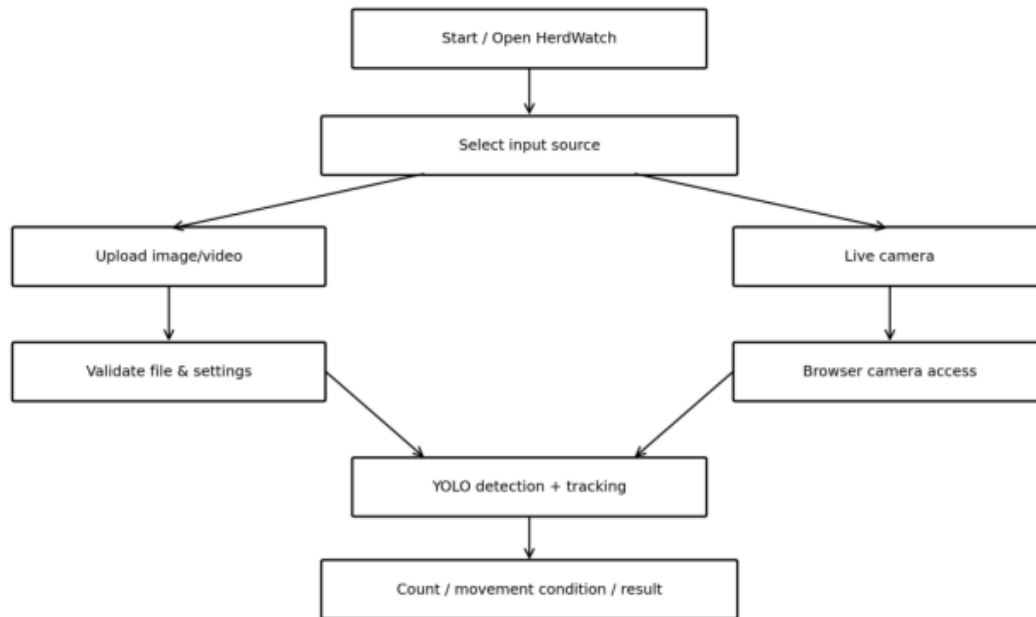


Figure 3.2: HerdWatch Process Flow

3.3 Main System Components

Component	Purpose
React Frontend	Dashboard, prediction screen, upload/live camera controls, ROI and line settings.
FastAPI Backend	API endpoints, prediction requests, WebSocket live processing, health checking.
YOLO11s	Detect cattle in each input frame.
ByteTrack	Track cattle between consecutive frames.
Counting Module	Calculate visible-in-ROI and directional line-crossing counts.
Movement Module	Estimate observable movement condition from tracking history.

3.4 Database Design: ER diagrams

The inspected application does not currently contain a relational database layer. However, a logical ER model is useful for documenting the information handled by the application and for guiding a future database implementation. The entities below correspond to the job/result structures already represented by the project.

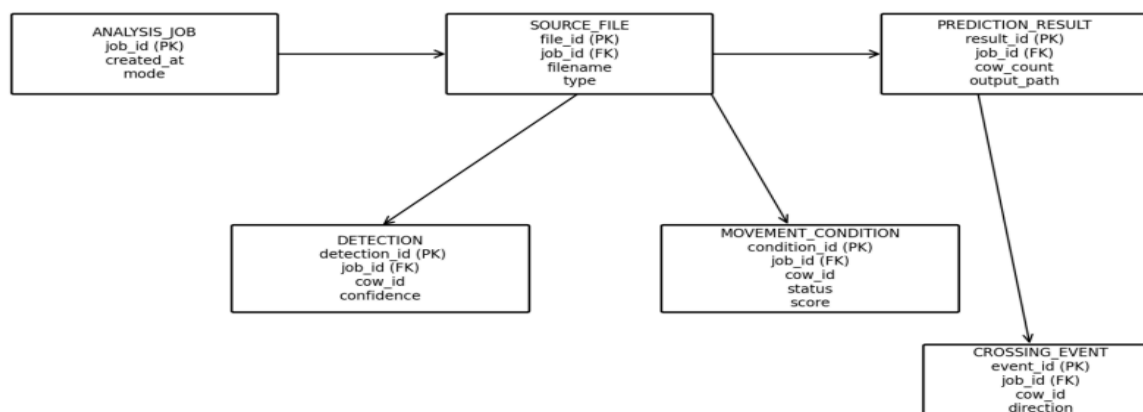


Figure 3.4: ER diagram

Entity	Important Attributes	Purpose
ANALYSIS_JOB	job_id, created_at, mode	Represents one monitoring/analysis session.
SOURCE_FILE	file_id, job_id, filename, type	Stores information about an uploaded source.
PREDICTION_RESULT	result_id, job_id, cow_count, output_path	Stores final analysis results.
DETECTION	detection_id, job_id, cow_id, confidence	Stores individual tracked detections.
MOVEMENT_CONDITION	condition_id, job_id, cow_id, status, score	Stores movement observations.
CROSSING_EVENT	event_id, job_id, cow_id, direction	Stores directional line-crossing events.

Relationship summary: one analysis job can have one source file and one or more result-related records. A job may produce multiple detections, movement observations, and crossing events. This model can later be implemented in MySQL, PostgreSQL, SQLite, or another persistent database if historical analytics are required.

4. Development Environment Setup

4.1 Tools and Technologies

Technology	Use in HerdWatch
Python	Backend and computer-vision processing
FastAPI	Backend API and WebSocket services
Uvicorn	Runs the FastAPI application
React	Frontend interface
React Router	Frontend navigation
Vite	Frontend build/development tooling
Tailwind CSS	Frontend layout and styling utilities.
Ultralytics YOLO	YOLO11s training, prediction and evaluation
OpenCV	Frame and image processing
NumPy	Numerical processing
WebSocket	Live camera frame communication
Pandas	Evaluation metric table/CSV generation.
Pillow	Dataset image validation and pixel hashing.
imageio / FFMPEG	Browser-compatible annotated video writing.

4.2 Development Environment

The repository is organized into a frontend application under `cv/frontend`, a FastAPI backend under `cv/backend`, and computer-vision/training code under `cv1/core` and `cv1/scripts`. The frontend package configuration specifies React 19.2.8, React Router 7.18.2, Vite 8.2.0 and Tailwind CSS 4.3.3. The backend requirements are maintained in `backend/requirements.txt`, while the vision code imports libraries such as Ultralytics, OpenCV, NumPy, Pandas and Pillow.

For local development, the frontend is configured to communicate with a backend through the local development environment. The FastAPI application enables CORS for `http://localhost:5173` and `http://127.0.0.1:5173`. The backend also mounts the project outputs directory at `/outputs` so generated images and videos can be accessed by the application.

4.3 Coding Standards

- Use descriptive module and function names such as `process_source`, `assess_track_condition` and `train_model`.
- Keep configuration validation close to the configuration object so invalid ROI or confidence settings fail early.
- Use type hints for core functions and data structures where practical.
- Separate training scripts from production prediction code.
- Write output artifacts to structured directories and use job IDs to avoid collisions between requests.
- Use comments/docstrings to explain non-obvious computer-vision decisions such as dense-tile detection and top-K frame counting.
- Treat visual movement results as observations and avoid diagnostic language in the user interface.

4.4 Model and Dataset Preparation

The training pipeline uses a cleaned cattle-counting dataset with one object class. The cleaning script removes missing/empty annotations, converts polygon annotations into detection boxes when necessary, removes duplicate source names and duplicate pixel-identical images, and rebuilds grouped 80/10/10 train/validation/test splits. The training script then trains YOLO11s for 50 epochs by default, with image size 640 and batch size 4, using deterministic seed 42 and moderate augmentation.

The supplied project also contains pseudo-labelling support and dataset validation utilities. These are useful development aids, but the available archive does not include the original dataset contents or a complete exported test-metric report. Consequently, this project book does not invent precision, recall or mAP values.

5. Screen Layout and Design

5.1 Dashboard

The Dashboard is the landing screen for system status and feature orientation. It presents a 'Live-ready cattle intelligence' message, a link to the Cattle Monitor, and status cards for the API and detection model. The dashboard also explains the three main capabilities: video tracking, live camera monitoring and condition signals.

The dashboard includes a condition guide with four states: Normal movement, Low movement, High activity and Observing. It also contains a prominent warning that the system reports visible movement patterns and detection confidence and cannot diagnose lameness, illness, pain, pregnancy or other medical conditions.

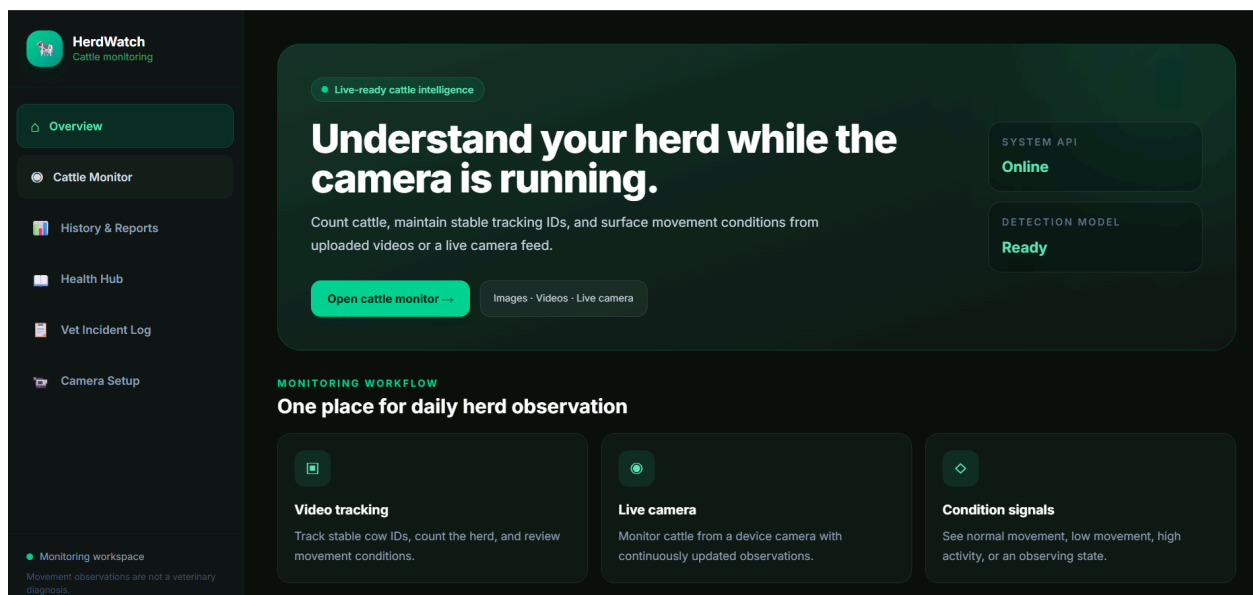


Figure 5.1. Dashboard Layout

5.2 Cattle Monitor Screen

The monitoring screen provides two operating modes: Upload Footage and Live Camera. It includes a Camera Counting Zone where the user can configure line orientation, line position, detection confidence, and ROI. The live view displays both the source camera stream and the AI tracking output.

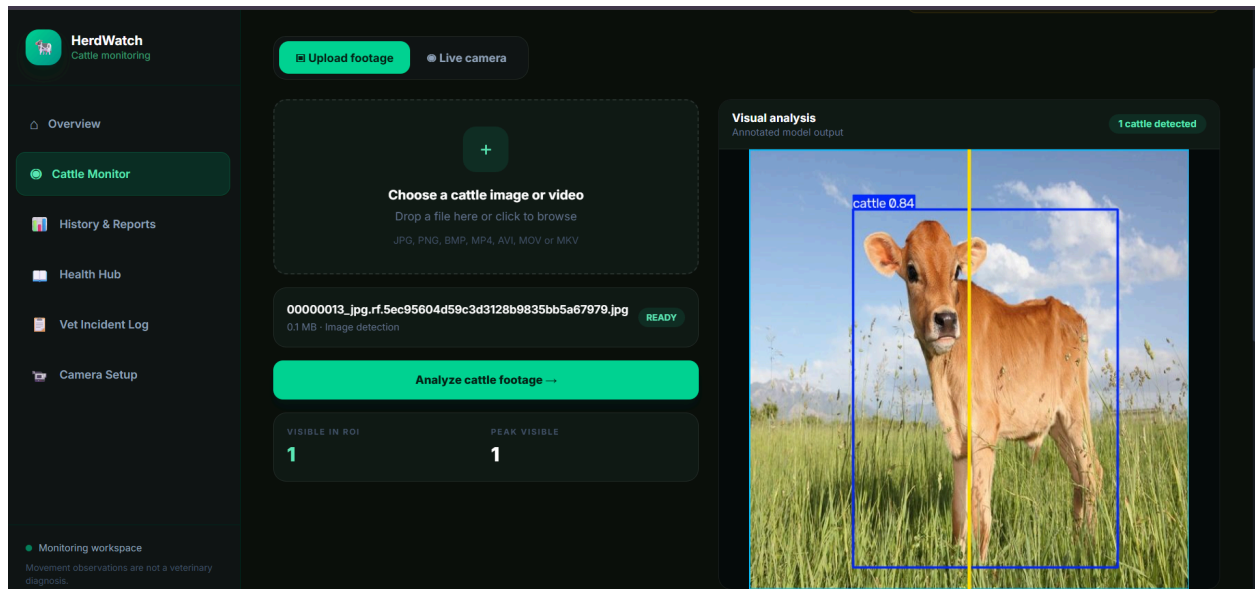


Figure 5.2. Monitor Screen Layout

5.3 Main Interface Functions

Screen/Control	Function
Dashboard	System overview, API status, model readiness, capability summary.
Upload Footage	Select an image/video for analysis.
Live Camera	Start/stop browser camera capture and live AI analysis.
Line Orientation	Choose vertical or horizontal counting movement.
Line Position	Move the counting line within the normalized scene.
Detection Confidence	Adjust the confidence threshold used by the monitoring session.
ROI	Restrict monitoring and counting to a selected region.
AI Tracking Output	Display annotated frames with detections/tracking.
Session Crossings	Show tracked crossing count.
Visible in ROI	Show cattle currently visible in the configured ROI.
Forward / Reverse	Show directional line-crossing counts.
Tracked Conditions	Show the number of condition observations.

5.4 Design Principles

- Keep the most important monitoring values visible without requiring navigation.
- Use clear controls for camera, ROI, line, and confidence configuration.
- Separate source imagery from AI-annotated output for easy visual comparison.
- Provide explicit status and error messages for unavailable camera/API/model resources.
- Keep animal-condition language observational and non-diagnostic.

6. Conclusion and Reflection

6.1 Project Summary

HerdWatch Vision demonstrates how a computer-vision model can be turned into an end-to-end monitoring application. The project does more than perform a single object-detection call: it adds configurable ROI filtering, stable video tracking, directional line crossing, movement-history analysis, a web API and a React user interface. This provides a foundation for practical cattle observation from common camera sources.

The supplied demonstration artifact successfully records two cattle detections from an image, and the annotated output shows the detector drawing bounding boxes and confidence labels. The runtime architecture is also structured so that a trained model can be reloaded when the model file changes.

6.2 Challenges and Solutions

Challenge	Solution Implemented
Cattle can appear close together and be difficult to separate.	The prediction module includes a dense-tile detection pass for likely tightly packed groups and applies non-maximum suppression.
Repeated video detections can cause over-counting.	ByteTrack maintains track IDs and the LineCrossingCounter records each counted track ID only once.
Small detection noise can create unstable line-crossing events.	The counter uses hysteresis, minimum track frames and an ROI boundary check.
A single noisy frame may inflate a video count.	The peak-frame count helper uses the median of the top-K per-frame counts rather than an absolute single-frame maximum.
Movement observations could be misunderstood as medical diagnoses.	The UI explicitly labels results as movement observations and includes a veterinary warning.
Different API jobs could overwrite the same output.	Prediction artifacts and metric files are stored using a job ID.
Browser playback can be sensitive to video encoding.	The video writer uses H.264 MP4 through imageio/FFMPEG for browser-compatible output.

6.3 Lessons Learned

- Object detection alone is not enough for reliable video counting; temporal tracking and event logic are important.
- Configuration such as ROI, confidence and line position should be exposed to users because camera views vary.
- Dataset cleaning and leakage-resistant splitting are essential before training an object detector.
- Safety-oriented wording matters when computer vision is applied to animal behaviour or welfare.
- Separating frontend, backend and core vision logic makes it easier to test and extend the application.
- Evaluation should distinguish a single demonstration result from measured dataset-level performance.

6.4 Future Enhancements

- Add a dedicated, validated pose/keypoint model if lameness or posture analysis is required.
- Add persistent database storage for sessions, historical counts and condition trends.
- Add authenticated multi-user access and role-based permissions for farm staff and administrators.
- Provide downloadable reports with daily/weekly herd counts and movement summaries.
- Add camera calibration and automatic ROI suggestions.
- Improve occlusion handling and evaluate tracker performance on crowded cattle scenes.
- Create a formal human-reviewed test set and report precision, recall, mAP@50 and mAP@50:95 alongside counting accuracy.
- Add automated alerts for persistent low movement, while retaining veterinary review as the final decision point.
- Add deployment documentation for edge devices or farm-local servers.

7. Appendices

Appendices A - Screen and Detection output

The following figure is a stored prediction artifact included in the project archive. It demonstrates the visual output format: cattle are enclosed by bounding boxes and the detector confidence is displayed. The associated job JSON reports two cattle detected with visible_in_roi counting.

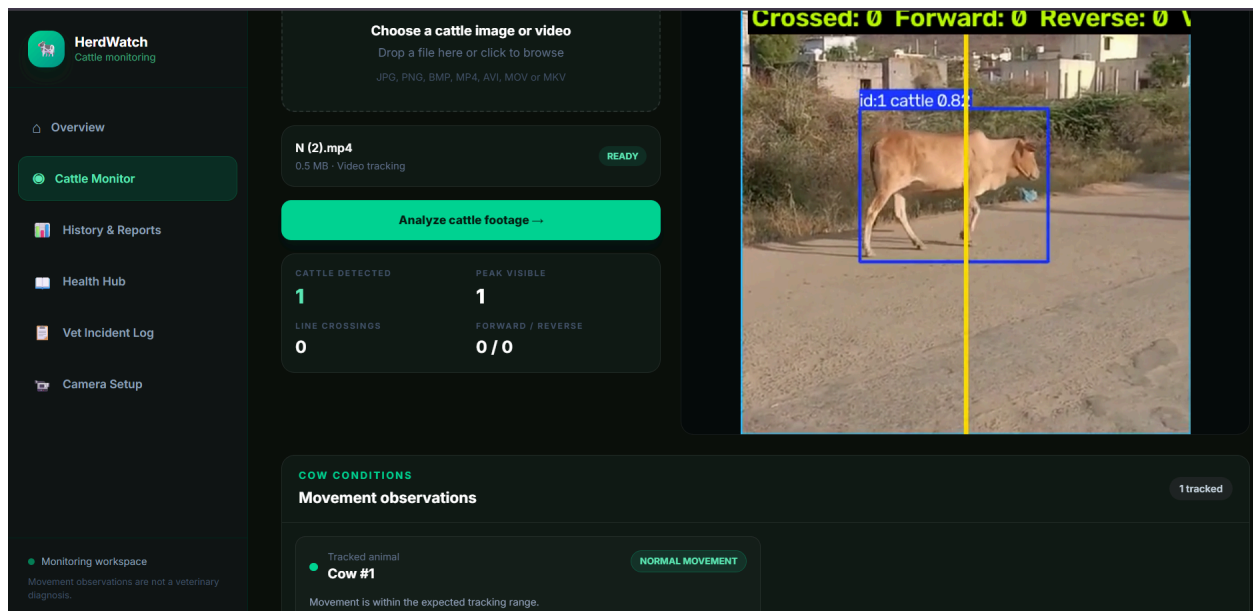


Figure 7.1. Detection Output Layout

Appendices B - Key Project Source Structure

Source file	Role
cv/backend/main.py	FastAPI application, CORS, static outputs, health endpoint, prediction router.
cv/backend/routers/predict.py	Image prediction and live webcam/WebSocket processing.
cv/frontend/src/App.jsx	React routing for Dashboard and Predict screens.
cv/frontend/src/pages/Dashboard.jsx	System overview and condition guidance.
cv/frontend/src/pages/Predict.jsx	Upload/live monitoring, camera capture, WebSocket, ROI and counting controls.
cv1/core/train.py	YOLO11s training configuration and model artifact export.
cv1/core/predict.py	Detection, tracking, counting, ROI and movement-condition logic.
cv1/core/evaluate.py	Test-set model evaluation and metric export.

Appendices C - References

1. Project source code: `cv1/core/train.py` – YOLO11s training pipeline.
2. Project source code: `cv1/core/predict.py` – detection, tracking, counting and movement observation pipeline.
3. Project source code: `cv1/core/lameness_classifier.py` – counting configuration and line-crossing logic.
4. Project source code: `cv1/core/evaluate.py` – test-set evaluation and metric export.
5. Project source code: `cv/backend/main.py` and `cv/backend/routers/` – FastAPI backend.
6. Project source code: `cv/frontend/src/pages/` and `cv/frontend/src/components/` – React interface.
7. Project artifact: `outputs/metrics/jobs/f3a6aa66db4c49d89fef749bd8031c74.json` – demonstration job metrics.
8. Project artifact: `outputs/predictions/f3a6aa66db4c49d89fef749bd8031c74.jpg` – demonstration annotated detection.